

Barman (Backup and Recovery Manager) — An Orientation for the Experienced PostgreSQL DBA

Reference document · Current as of July 4, 2026

TL;DR

- **Barman is the actively-maintained, GPLv3, Python backup/DR manager for PostgreSQL, created by 2ndQuadrant in 2011 and maintained by EDB since its 2020 acquisition of 2ndQuadrant.** The current stable release is **3.19.1 (26 May 2026)**; it targets a "backup server pulls from many Postgres servers" model and now supports PostgreSQL 17/18 native block-level incremental backups.
- For an expert DBA, the mental model is straightforward: Barman is a catalog + orchestration layer over the primitives you already know (`pg_basebackup`, `pg_receivewal`, WAL archiving, PITR). The key decisions are **backup method** (`postgres`/streaming vs `rsync`/SSH) and **WAL transport** (`streaming_archiver` via `pg_receivewal` + replication slot, vs file-based `archiver` via `archive_command`).
- Barman is the pragmatic default for **centralized, multi-server, on-prem/enterprise** backup with EDB commercial support behind it; **barman-cloud** and the **CloudNativePG Barman Cloud plugin** cover object-storage and Kubernetes. For raw block-level incremental/parallel-restore performance on very large single clusters, pgBackRest historically remains stronger — a comparison the DBA should make deliberately.

Key Findings

What Barman is and who maintains it

Barman ("Backup and Recovery Manager") is an open-source administration tool for remote backups and disaster recovery of PostgreSQL, written in Python and distributed under **GNU GPL v3**. It was started by **2ndQuadrant in 2011**; **EDB (EnterpriseDB) acquired 2ndQuadrant in 2020** and now maintains the project. It runs on Linux/Unix only and operates from a dedicated backup host, remotely managing a catalog of backups plus PITR for many Postgres servers from one location.

The current lead maintainers/core contributors at EDB include **Martín Marqués** (Director of Engineering for Database Tools, leads the Barman team), **Giulio Calacoci** (Principal Engineer, long-time Barman contributor), and **Israel Barth Rubio**. The canonical repo moved from SourceForge to GitHub under (`EnterpriseDB/barman`) (starting with 2.13).

Version and release cadence

- Current stable: **3.19.1**, released **26 May 2026** (per `pgbarman.org` and the docs release notes).
- Cadence: frequent minor/patch releases — several per year. In roughly the last year the 3.11 → 3.19 line shipped: 3.11 (PG17 incremental), 3.13/3.14 (WORM/immutable + GPG encryption), **3.15 announced 26 Feb 2026**, 3.16–3.17, **3.18.0 (whose announcement highlights block-level incremental backups for cloud storage and the new `backup_method = local-to-cloud`)**, and 3.19 (May 2026).
- Minimum supported Postgres: version 11+ (PG10 and earlier dropped in the 3.x line; 9.6 and earlier dropped at 3.0).

Core architecture and concepts

- **Backup server model.** A Barman host pulls backups and WAL from one or more Postgres servers; a single Barman install manages many servers, each with its own config, catalog, and retention.
- **Backup methods** (`backup_method`):
 - `postgres` — streaming backups via `pg_basebackup`; required for PG17+ native block-level incrementals. No SSH needed.
 - `rsync` — rsync-over-SSH; supports Barman's classic file-level incremental (hard-link/`reuse_backup`) and parallel copy (`-j`).
 - `snapshot` — cloud volume snapshots (AWS EBS, GCP disks, Azure managed disks) with Barman as the WAL/catalog store.
 - `local-to-cloud` (introduced 3.18/3.19) — direct cloud backups configured through the main `barman` CLI with an object-store URL.
- **WAL transport, two mechanisms:**
 - **WAL streaming** via `pg_receivewal` (enable `streaming_archiver = on`), strongly recommended with a **replication slot** (`slot_name`), plus `create_slot = auto` to guarantee no WAL is removed before Barman receives it. Lowers RPO.
 - **WAL archiving** (file-based) via Postgres `archive_command`, typically calling `barman-wal-archive` (the robust replacement for a raw `rsync/scp archive_command`). Controlled by `archiver = on`.
 - The two can be combined; if neither is explicitly set, Barman defaults `archiver` on for back-compat.
- **Client utilities:** `barman-wal-archive` (used in `archive_command`) and `barman-wal-restore` (used in `restore_command`) on a standby/recovery node) ship in the `barman-cli` package.

- **barman-cloud-* utilities** (`barman-cli-cloud` package): (`barman-cloud-backup`), (`-wal-archive`), (`-restore`), (`-wal-restore`), (`-backup-list`), (`-backup-show`), (`-backup-delete`), (`-backup-keep`). Support **AWS S3 (and S3-compatible), Azure Blob Storage, and Google Cloud Storage**. These can run standalone on the Postgres host (no Barman server needed) or be wired into hook scripts to relay a Barman backup to the cloud.
- **Incremental backups, two lineages:**
 - Classic Barman file-level incremental (`rsync + (reuse_backup = link)`), around since pre-2.0.
 - **PostgreSQL 17+ native block-level incremental** via (`pg_basebackup`), **supported since Barman 3.11** (`backup_method = postgres`), (`--incremental`), (`latest-full`) shortcut). Barman 3.11 also renamed the (`barman show-backup`) field "Incremental size" to "Resources saving" (`resource_savings` in JSON). Requires (`summarize_wal = on`) on the server (WAL summarization). Restore combines the chain with (`pg_combinebackup`), staged locally (`(local_staging_path)/(staging_path)`). Compressed backups are not supported for block-level incrementals.
- **Retention policies:** (`retention_policy`) by (`RECOVERY WINDOW OF n DAYS/WEEKS`) or (`REDUNDANCY n`); (`minimum_redundancy`) to protect a floor of backups; note that only **full** backups count toward retention because incrementals need their full parent chain.
- **Hook scripts:** (`pre_/post_backup_script`) (and (`_retry_`) variants), (`pre_/post_archive_script`), etc. — commonly used to relay to cloud/off-site (the 3-2-1 rule) or to run (`barman-cloud-backup`).
- **Geo-redundancy:** a passive Barman node can synchronize from a primary Barman node (introduced 2.6) for a second backup tier in another DC/AZ.
- **Recovery / PITR:** (`barman recover`) (aka restore) rebuilds a data directory (locally or remotely over SSH) and sets recovery targets (`(--target-time`, `--target-xid`, `--target-lsn`, `--target-name`, `timeline)`). Barman writes the appropriate recovery configuration/`(recovery.signal)`.
- **Monitoring:** (`barman check <server>`) runs a battery of health checks (WAL archiving, replication slot, (`receive-wal`) running, retention, minimum redundancy, (`pg_basebackup`)/(`pg_receivewal`) compatibility, etc.) and is the command you wire into monitoring; (`barman diagnose`) emits a JSON report of all servers for support. (`barman cron`) (run every minute; installed by the packages) drives maintenance, starts (`receive-wal`), and enforces retention.

Key configuration concepts

- Global config (`/etc/barman.conf`); per-server configs in (`/etc/barman.d/<server>.conf`) (`(.conf)` suffix). (The Fedora/EPEL packages instead use (`/etc/barman/barman.conf`) and (`/etc/barman/conf.d`) — do not mix EPEL and PGDG/EDB packages.) (`Pgbarman`)
- Essential per-server keys: (`conninfo`), (`streaming_conninfo`), (`backup_method`), (`streaming_archiver`), (`slot_name`), (`create_slot`), (`archiver`), (`path_prefix`) (points at correct client binaries per PG version), (`retention_policy`), (`minimum_redundancy`), (`reuse_backup`), plus staging options for incremental/encrypted restore (`(staging_path)`, (`staging_location`), unified in 3.19; formerly (`local_staging_path`)/(`recovery_staging_path`)).
- Two Postgres roles are typical: a (`barman`) role (maintenance/backup) and a (`streaming_barman`) role with **REPLICATION** for (`pg_basebackup`)/(`pg_receivewal`).

Notable recent developments (3.x)

- **PG17+ native incremental backup** support (Barman 3.11).
- **GPG encryption** of tar backups and WAL files (Barman 3.14; (`encryption`), (`encryption_key_id`); encrypted files get (`.gpg`); requires compression; decrypt-on-restore). **Barman 3.14.1 fixed a WAL-archiving performance regression** — an oversight in the 3.14 encryption logic that caused Barman "to check for encryption in WAL files streamed to the server, which is unnecessary as streamed WALs are never GPG-encrypted, nor compressed."
- **WORM / immutable storage** support: (`worm_mode`) (3.13.x) for on-disk immutable/WORM targets (relocates (`backup.info`)/annotations to a (`meta`) dir; requires a grace period, ~15 min recommended; (`barman keep`) unsupported in WORM); **AWS EBS Snapshot Lock** (3.13) and **S3 Object Lock** verification (`(--check-object-lock)`, (`aws_check_object_lock`) in 3.19) for cloud WORM.
- **local-to-cloud backup method**, unified (`staging_path`)/(`staging_location`), (`barman export-backup`)/(`import-backup`), cloud block-level incremental, and cloud restore via (`barman cloud-wal-restore`) (3.18/3.19).
- **Known gotcha:** a regression in **barman-cli-cloud 3.19.x** double-writes (`backup.info`) and **fails against retention/Object-Lock-protected destinations** (GitHub issue #1195); affected users pinned to 3.18.0 until fixed. Verify your version if you use cloud WORM.

Comparison vs alternatives (brief, factual)

- **pgBackRest** (C; block-level incremental, parallel backup/restore, (`--delta`)): historically the performance leader for multi-TB single clusters and the community default. Note the 2026 turbulence: maintainer David Steele posted a "NOTICE OF OBSOLESCENCE" and archived the repo on **27 April 2026** (citing the Crunchy Data → Snowflake sale; the final solo-maintainer release, **v2.58.0, shipped 19**

January 2026). It was then **revived on 18 May 2026** — the GitHub repo was unarchived and a sponsor coalition (**AWS, Supabase, pgEdge, Tiger Data, Percona, and Eon.io**, with additional sponsors Xata, Dalibo, and Data Egret) funded ongoing work, with David back at work. During the archived window many practitioners (e.g., Christophe Pettus, mydbanotebook) named Barman the leading actively-maintained alternative.

- **WAL-G** (Go; cloud-native, delta backups, parallel, multi-DB): simplest infra, best for cloud/Kubernetes and small teams; weaker on centralized multi-server management and documentation.
- **pg_probackup** (Postgres Pro; synthetic full backups, advanced incremental): strong incremental story, smaller community.
- **When to choose Barman**: centralized management of many servers from one host; enterprise/EDB commercial support; a Python codebase your team can read/extend via hooks; PG17+ native incremental with catalog/retention; cloud via barman-cloud; Kubernetes via CloudNativePG's Barman Cloud plugin. Consider pgBackRest instead when block-level incremental and fast parallel/delta restore on very large single clusters dominate your RPO/RTO math.

Barman and CloudNativePG (Kubernetes)

CloudNativePG (the CNCF PostgreSQL operator, created by EDB/Gabriele Bartolini) uses **Barman Cloud under the hood** for object-storage backup/WAL archiving. The historical **in-tree** (`barmanObjectStore`) support is **deprecated** and being replaced by the standalone **Barman Cloud Plugin** (via the CNPG-I plugin interface, using an `ObjectStore` CRD). Deprecation began in **CNPG 1.26**; the removal date for in-tree support has slipped over time (from 1.28 → 1.29 → 1.30 → now **1.31.0**, confirmed verbatim in the CloudNativePG release notes: "Updated the deprecation notice for native (in-tree) Barman Cloud support to reflect that it will now be removed in CloudNativePG 1.31.0, rather than 1.30.0"). Backups made with the in-tree approach remain compatible with the plugin, and a migration guide exists. For new deployments, use the plugin.

Details

Official resources

- **Documentation**: <https://docs.pgbarman.org/> (versioned; user guide, configuration reference, release notes, FAQ). Project site: <https://pgbarman.org/> (shows current version + release-notes link; downloads at <https://pgbarman.org/downloads/>; support at <https://pgbarman.org/support/>).
- **Source code**: <https://github.com/EnterpriseDB/barman> (GPLv3). Releases/changelog: <https://github.com/EnterpriseDB/barman/releases> and (`RELNOTES.md`).
- **PyPI**: <https://pypi.org/project/barman/> (extras: `cloud`, `aws-snapshots`, `azure`, `google`, `lz4`, `snappy`, `zstandard`, etc.). `pip install barman` also installs the barman-cloud scripts.
- **Packages**: PGDG apt/yum repos (packaged with each OS's default Python); EDB repositories; also a Barman-maintained public APT/RPM repo. **Avoid mixing EPEL and PGDG/EDB packages** (different config layouts).
- **Community support**: the (`pgbarman`) Google Group (<https://groups.google.com/g/pgbarman>) and the GitHub issue tracker; EDB offers 24×7 commercial support.
- **Release announcements**: posted to [postgresql.org/about/news](https://www.postgresql.org/about/news) and mirrored in the docs' release-notes section.

Top ~20 articles, talks, and tutorials on Barman

Foundational / official

1. **Barman Manual (official docs)** — EDB/Barman team — docs.pgbarman.org — The authoritative reference; read the "Concepts/Before you start," "Quick start," Backups, Recovery, and Configuration chapters first.
2. **"Barman, past, present, and future of a pioneer community project"** — Giulio Calacoci & Martín Marqués — PGConf.EU 2025 (Riga), session #7103 — <https://www.postgresql.eu/events/pgconfeu2025/sessions/session/7103/> (slides PDF linked there; recording on the PostgreSQL Europe YouTube channel) — The current maintainers' state-of-the-project retrospective and roadmap; the single best recent talk.
3. **EDB Docs — Barman (single-server streaming walkthrough)** — EDB — enterprisedb.com/docs/supported-open-source/barman/ — Hands-on config of a streaming-only server (`backup_method = postgres`, `streaming_archiver`, `slot_name`, `create_slot = auto`) plus the (`barman check`) / (`switch-wal`) workflow.

Cloud (barman-cloud) 4. **"Barman Cloud – Part 1: WAL Archive"** — Jonathan Battiato, EDB/2ndQuadrant — enterprisedb.com/blog/barman-cloud-part-1-wal-archive — (`barman-cloud-wal-archive`) as (`archive_command`) / hook script to S3. 5. **"Barman Cloud – Part 2: Cloud Backup"** — EDB — enterprisedb.com/blog/barman-cloud-part-2-cloud-backup — (`barman-cloud-backup`) internals, parallel upload, encryption. 6. **"Barman 2.11: barman-cloud-restore and barman-cloud-wal-restore"** — EDB — enterprisedb.com/blog/barman-211-barman-cloud-restore-and-barman-cloud-wal-restore — The recovery side of the cloud story. 7. **"Using Barman Cloud Utilities for Backups and WAL Archiving in PostgreSQL"** — Percona — percona.com/blog/using-barman-cloud-utilities-for-backups-and-wal-archiving-in-postgresql/ — Vendor-neutral, PG17-era walkthrough for S3/GCS incl. (`barman-cloud-backup-show`). 8. **"How to Backup Your PostgreSQL to Azure Blob Storage with**

Barman — EDB — enterprisedb.com/blog/BARMAN-Backup-Azure-Blob — Azure-specific barman-cloud tutorial. 9. **"How to restore backups of PostgreSQL from S3 using Barman"** — blog.cynexia.com — A concise, real restore runbook (arch/version match, `barman-cloud-restore`), WAL replay).

Incremental backup (PG17/18) 10. **"Why PostgreSQL 17's Incremental Backup Feature is a Game-Changer"** — EDB — enterprisedb.com/blog/why-postgresql-17s-incremental-backup-feature-game-changer — How Barman 3.11 shipped PG17 incremental support and the "Resources saving" metric. 11. **"Incremental Backup Challenges and Solutions for PostgreSQL 17"** — Robert Haas, EDB — enterprisedb.com/blog/fine-tuning-incremental-backup — The `pg_combinebackup` staging problem and the PG18 `--link` fix (by Israel Barth Rubio) Barman will use; essential deep dive. 12. **Barman 3.11 release announcement** — postgresql.org/about/news — Primary source for the exact incremental CLI/behavior (`--incremental`, `latest-full`, `local_staging_path`).

Comparisons 13. **"After pgBackRest — the build"** — Christophe Pettus (thebuild.com, 30 Apr 2026) — Sober axis-by-axis comparison; blunt about where Barman is / isn't a pgBackRest substitute. 14. **"pgBackRest is dead. Now what?"** — mydbanotebook.org — Recommends Barman as the serious actively-maintained alternative (with a 19 May 2026 update noting pgBackRest's revival). 15. **"PostgreSQL Backup & Recovery Part 5 — pgBackRest vs Barman vs WAL-G"** — DBLog (dblog.co.kr) — A 2026 decision matrix by DB size/topology/team. 16. **"Automating Backups and DR in PostgreSQL at Scale: pgBackRest vs. Barman"** — Severalnines — Operational trade-offs at scale.

Kubernetes / CloudNativePG 17. **"Migrating from Built-in CloudNativePG Backup"** — CloudNativePG docs — cloudnative-pg.io/plugin-barman-cloud/docs/next/migration/ — The canonical in-tree → Barman Cloud plugin migration guide (ObjectStore CRD). 18. **"Exploration: PostgreSQL Cluster Backup with Barman (CNPG)"** — Umut Tekin, CYBERTEC — cybertec-postgresql.com — End-to-end CNPG + Barman Cloud plugin + MinIO backup/restore demo. 19. **"Deployment of PostgreSQL Replica Cluster via Barman Cloud Plugin on CloudNativePG"** — EDB — enterprisedb.com/blog — Plugin-based backup/WAL archiving across primary/replica clusters (cert-manager, ObjectStore).

Classic overviews / historical 20. **"Using Barman to Backup PostgreSQL - An Overview"** — Severalnines — Good architectural overview and RMAN analogy. 21. **"PG Phriday: Adventures in BAR Management"** — Shaun Thomas, EDB — enterprisedb.com/blog/pg-phriday-adventures-bar-management — Hybrid rsync-data + `pg_receivewal`-WAL setup and the 3-2-1 rule. 22. **"Combining Barman and EDB Postgres Advanced Server"** — Stefan Fercot (pgstef.github.io) — Clean dedicated-Barman-server + 3-node streaming cluster build. 23. **"PostgreSQL continuous backup and PITR with Barman" (webinar) / "PostgreSQL Disaster Recovery with Barman" (PGConf.EU 2013)** — Gabriele Bartolini (Barman's original lead) — slideshare.net/GabrieleBartolini — Conceptual foundations from the project founder; the DR-culture framing still holds.

Other relevant material

- **Books:** *Learn PostgreSQL* (2nd ed., Luca Ferrari & Enrico Pirozzi, Packt, PG16) includes a dedicated disaster-recovery chapter covering backup tooling. General PostgreSQL administration/HA titles typically reference Barman in their backup/DR chapters. The docs remain the primary reference for Barman specifics.
- **CloudNativePG relationship (summary):** CNPG uses barman-cloud; new deployments should use the **Barman Cloud Plugin** (CNPG-I) with an `ObjectStore` resource; in-tree `barmanObjectStore` is deprecated (since 1.26) and slated for removal in CNPG 1.31.0. When on the plugin, some in-tree Cluster `.status` backup fields/metrics stop updating — adjust observability accordingly.

Common gotchas & operational best practices (for a newcomer to Barman)

- Use `streaming_archiver = on` + a replication slot (`slot_name` + `create_slot = auto`) so Postgres retains WAL until Barman receives it; this is the low-RPO, no-SSH path. Without a slot you're back to `wal_keep_size` guesswork.
- `barman check` will show **"WAL archive FAILED" right after setup** simply because no WAL has arrived yet — force a segment switch with `barman switch-wal --force --archive` (or `SELECT pg_switch_wal()`), then re-check. Wire `barman check` into monitoring.
- Run `barman cron` every minute (the packages install this) — it starts `receive-wal`, processes WAL, and enforces retention.
- Set `minimum_redundancy` (>0) so retention never deletes your last safety net; remember only full backups count toward retention when using incrementals.
- **PG17 incremental needs** `summarize_wal = on` on the server *before* the initial full backup; incrementals require `backup_method = postgres`; compressed block-level incrementals aren't supported; take a fresh full if you change `data_checksums`.
- **Test recovery regularly** — a catalog of backups you've never restored is not a DR plan; use `barman recover` to a scratch host and validate.
- When `backup_method = postgres`, Barman doesn't SSH to the server, so external config files aren't captured — use a post-backup hook + `barman show-server` to grab them.
- **Cloud WORM caution:** verify your barman-cli-cloud version — 3.19.x has a `backup.info` double-write regression (issue #1195) that fails backups against retention/Object-Lock-protected buckets; pin to 3.18.0 until fixed. For on-disk `worm_mode`, provide a grace period and remember `barman keep` isn't supported there.
- **Don't mix package sources** (EPEL vs PGDG/EDB) — different config-file layouts can break upgrades.

1. **Stand up a lab first (streaming-only).** On a dedicated Barman host, install from PGDG/EDB, create `(barman)` + `(streaming_barman)` roles, and configure one server with `(backup_method = postgres)`, `(streaming_archiver = on)`, `(slot_name = barman)`, `(create_slot = auto)`. Run `(barman check)`, force a WAL switch, take `(barman backup)`, then **practice `(barman recover)` to a throwaway host**. This exercises 90% of daily operations. *Benchmark to proceed:* a clean `(barman check)` and a successful PITR to a target time.
2. **Decide backup method by scale and topology.** Default to `(postgres)`/streaming for simplicity and PG17+ incrementals. Choose `(rsync)` if you need Barman's file-level dedup/parallel copy or remote-server nuances. *Threshold to reconsider Barman entirely:* if you have a single multi-TB, write-heavy cluster where backup windows and restore RTO hinge on block-level delta restore, benchmark pgBackRest before committing.
3. **Design WAL safety and retention explicitly.** Always use a replication slot; set `(retention_policy)` (recovery window) and `(minimum_redundancy)`; enable `(summarize_wal)` if you'll use native incrementals.
4. **Add a second tier.** Use `(post_backup)` hooks or barman-cloud to push copies to S3/Azure/GCS (3-2-1), or configure a passive Barman node for geo-redundancy. For regulated/ransomware-sensitive shops, evaluate WORM (`(worm_mode)`, EBS Snapshot Lock, S3 Object Lock) — but pin around the 3.19.x cloud-WORM regression.
5. **On Kubernetes, go plugin-first.** For CloudNativePG, adopt the Barman Cloud Plugin (ObjectStore CRD) for all new clusters and plan migration off in-tree `(barmanObjectStore)` before CNPG 1.31.0.
6. **Operationalize monitoring and restore drills.** Put `(barman check)` in your monitoring, schedule periodic restore tests, and keep `(barman diagnose)` output handy for support.

Caveats

- **Version currency:** 3.19.1 (26 May 2026) is confirmed via pgbarman.org and the docs. Barman releases often; re-check the release notes before deploying, especially given the 3.19.x cloud-WORM `(backup.info)` regression (GitHub issue #1195), which was still being tracked and may be fixed in a later patch by the time you read this.
- **pgBackRest status is a moving target:** it was archived (27 Apr 2026) then revived (18 May 2026) by a sponsor coalition. Comparison articles written inside that window vary in tone; treat "pgBackRest is dead" framing as time-bound. Both tools are actively maintained as of July 2026.
- **Team composition:** Martín Marqués, Giulio Calacoci, and Israel Barth Rubio are confirmed EDB Barman leads/contributors; other names sometimes cited (e.g., Gustavo Bagatini, Barbara Leidens) appear in release credits but weren't independently confirmed as core maintainers here.
- **Some cited tutorials are older** (PG12-era barman-cloud posts, 2013 Bartolini slides) — concepts are sound but command flags and defaults (e.g., exclusive backups, `(pg_receivexlog)`→`(pg_receivewal)`, provider support) have since changed; cross-check against current docs.
- **A few comparison sources are SEO/aggregator-grade** (dev.to, Medium); they're included for breadth but the authoritative claims here rest on docs.pgbarman.org, postgresql.org news, EDB/CNPG official blogs, and named practitioners (Pettus, Haas, Fercot).